

面试准备文档 — 宅家激励 + 嘴替

面向岗位：AI 应用开发 | 身份：工程造价应届生

一、宅家激励 App

Q1：AI 推荐引擎的工作流程？

五步流水线：

获取天气 (Open-Meteo, 缓存30min) + GPS定位 (缓存30min)

→ POI搜索 (Overpass API, radius=3000m, 并行搜自然+商业两类)

→ 构建Prompt (System角色设定 + User 8字段拼接)

→ 调用DeepSeek API (maxTokens=800, temp=0.8, 超时8s)

→ 解析响应 (JSON.parse → 正则提取 → 兜底默认值)

三级降级： AI 直调 → 本地规则引擎 → 预设任务库随机

追问"为什么用 LLM 而不用纯规则？"

规则只能硬匹配（下雨=取消户外），LLM 能软决策（小雨+用户爱散步=推荐带伞出门）。线上可用规则粗筛 + LLM 精排降低成本。

追问"路线校验怎么做？"

- stops 必须非空数组
- 每站自动补全 order/name/category/aiReason/walkingFromPrev/suggestedDuration
- 总耗时校验：

sum(站间步行 + 停留) + 最后一站到家

 ≤ maxMinutes
- 步行速度常量：80m/min (4.8 km/h)

追问"AI 任务积分怎么算? "

`points = 50 + stops.length × 10` (基础50分 + 每站10分)

Q2: Prompt 怎么设计的?

System Prompt (约30行中文) :

- 角色: "出行规划师"
- 输出约束: 2-4站环形路线、站间距5-20分钟步行、严格JSON、禁止markdown包裹

User Prompt (8字段拼接) :

当前时间 + 天气 + 位置(城市+坐标) + 兴趣 + 活动风格 + 最大步行时长 + 宠物(Lv/心情) + 周边地点(最多10条)

解析容错 (三层回退) :

1. `JSON.parse(content)` 直接解析
 2. 正则 ````json...```` 代码块提取
 3. 正则 `{[\s\S]*}` 提取对象字面量
-

Q3：天气模块的推荐逻辑？

条件	判定阈值	推荐倾向
雷暴	WMO码 95-99	禁止户外，推室内
高温	>35℃	推室内/傍晚
低温	<5℃	推室内
阵雨	WMO码 80-82	推室内
雨	WMO码 51-65	带伞+短途户外
雪	WMO码 71-77	室内为主
大风	>30km/h	不推荐户外

缓存30分钟，过期后网络失败可回退到过期缓存。

Q4: POI搜索的实现细节?

- 源: OSM Overpass API (免费, 无需Key)
 - 两轮并行: 自然类 (park/viewpoint/garden/square) 搜满3000m; 商业类 (11种) 只搜1800m (60%半径)
 - 去重: 按OSM ID + 名称 (大小写不敏感) 两层去重
 - 排序: 按用户偏好类别分级, 截断到15个
 - 距离格式化: $\geq 1000\text{m}$ 显示公里 (1位小数), $< 1000\text{m}$ 显示米
-

Q5：宠物养成数值怎么设计的？

参数	数值	设计意图
升级门槛	growth ≥ 100	约5-7个任务升一级
等级上限	10	4阶段（0.8x/1.0x/1.2x/1.4x倍率）
心情衰减	每天-15	必须每天登录互动
喂食	消耗20积分，心情+20	积分换心情
玩耍	消耗10积分，成长+10	积分换成长
积分获得	简单30/中等50/困难80	对应成长10/20/35
心情分级	≥80开心 / ≥60一般 / ≥40不开心 / ≥20难过 / <20生病	5级梯度
低心情惩罚	心情<30时触发率+30%	负面随机事件

追问"数值为什么这么设？"

每天登录做1-2个任务才能维持宠物心情，一周左右到满级。不会太快（失去目标）也不会太慢（流失）。

Q6：随机事件系统？

12种事件，触发率约85%。公式：`基础40% + moodModifier`（低心情+30%，高心情-10%）。天气感知：检测到晴/雨/雪/风时匹配对应话语。

Q7：每日重置怎么实现的？

- `lastRefreshDate`：跨天时重置免费刷新次数(3次) + 重新抽取每日任务
 - `lastMoodDecay`：心情 $\text{-= diffDays} \times 15$ ，归零不赋负
 - 每次App启动调用 `checkDailyReset()`
-

Q8: 为什么 PWA → Capacitor?

PWA 开发调试快（浏览器热刷新），但国内 Android 生态受限（微信内置浏览器不支持SW、无法通知）。Capacitor 复用100% Web代码打包APK，获得原生能力。

Q9: GPS定位失败怎么处理?

四级优先级: 缓存坐标 (TTL 30min) → 浏览器Geolocation(超时10s, 低精度) → Capacitor Geolocation → 手动输入城市

反向地理编码用 OSM Nominatim (免费, 语言zh) , 正向编码用 Open-Meteo Geocoding。

二、嘴替 App

Q10: 完整的 Prompt 设计?

【系统提示】

你是一个专业的微信回复助手，名字叫“嘴替”。

回复风格: {SHARP/HUMOROUS/PASSIVE/RATIONAL/GENTLE/DOMINEER}

反驳对象: {targetPerson}

核心立场: {coreIdeology}

规则: 50-150字、第一人称、中文、口语化、不攻击性太强、直接输出不加说明

【上下文】

{最近N条对话记录}

【当前消息】

{senderName} 说: "{content}"

追问“为什么风格放系统提示而不是用户提示？”

风格是持久角色设定，放系统提示模型更稳定遵守；用户提示放变化的上下文和当前消息。符合 OpenAI 官方最佳实践。

Q11：三级降级的具体实现？

第1级：有Key + 网络正常 → DeepSeek API → 真实回复

第2级：有Key + API失败(超时/限流/服务错误) → catch异常 → Mock

第3级：无Key → 直接Mock（不尝试网络）

API参数：model= `deepseek-chat` , max_tokens=500, temperature=0.8, 超时15s (AbortController)

Mock引擎不是简单随机：先 `detectKeywords` 正则匹配5类场景（催婚/借钱/工作/身材/关系），再从对应风格的5-8条模板中随机一条。模拟延迟600-1400ms让UI体验一致。

Q12：状态机的完整设计？



事件	触发条件	副作用
NEW_MESSAGE	收到微信通知	SAVE_TO_STORAGE, SHOW_PANEL
START_REBUTTAL	用户点击生成	CALL_AI
AI_RESPONSE	API返回	UPDATE_UI
REGENERATE	用户点重新生成	CALL_AI (计数+1)
EDIT_REBUTTAL	用户编辑文本	UPDATE_DRAFT
SEND_REBUTTAL	用户确认复制	COPY_TO_CLIPBOARD, ADD_TO_HISTORY
DISMISS	用户点忽略	REMOVE_FROM_QUEUE
ACKNOWLEDGE	用户点完成	RETURN_TO_IDLE

每个状态有守卫：当前状态不允许的事件返回 INVALID_TRANSITION 。

Q13: sideEffects 模式为什么重要?

核心思想：状态机只管"能不能转"，不管"转了之后干什么"。

```
// 状态机只返回意图
transition(session, START_REBUTTAL)
  → { session: {...}, sideEffects: ['CALL_AI'] }

// app.js 负责执行
switch (effect) {
  case 'CALL_AI': await RebuttalEngine.generate(...)
}
```

好处：状态机可独立单测（不依赖DOM/网络）、新增副作用不影响核心逻辑、全链路可追踪。

追问"这和Redux middleware有什么区别？"

思想类似但更轻量。Redux用middleware链，我用sideEffects数组+switch-case。项目规模小，手写更灵活且零依赖。

Q14: 双通道通知抓取架构?

```

AccessibilityService (主力)
├── TYPE_NOTIFICATION_STATE_CHANGED
├── event.getText() → 提取标题+内容
├── ParcelableData → Notification对象 (兜底)
└── → NotificationMessageQueue (静态线程安全队列)

```

```

NotificationListenerService (备用)
├── onNotificationPosted → 过滤com.tencent.mm
├── extras提取title/text/subText/bigText
├── startForeground() 常驻通知保活 (Android 14+)
└── → 同一 NotificationMessageQueue

```

```

ZuiTiNotificationPlugin (轮询层)
├── 500ms轮询 pollNewMessages()
├── notifyListeners("wechatMessage", data) → WebView
└── → NotificationBridge → App.handleWeChatMessage()

```

追问"为什么两个通道都用?"

国产ROM对不同服务类型限制策略不同。小米可能杀NotificationListener但保留AccessibilityService, 华为可能反过来。双通道是在小米和华为真机上复现问题后的经验结论。

追问"为什么用轮询而不是EventBus?"

Service和Plugin运行在不同上下文, 静态队列是最可靠的跨上下文通信方式。EventBus需要共享进程内引用, 服务被杀后状态丢失。

Q15：通知去重算法？

微信同一条消息可能触发多条通知（锁屏+横幅+通知中心）。

```
// Jaccard相似度：交集词数 / 并集词数  
// 条件：同一发送者 + 相似度  $\geq$  70% + 3秒时间窗口  $\rightarrow$  丢弃
```

举例："你在干嘛呢" vs "你在干嘛呢" $\rightarrow$ 100%相似 $\rightarrow$ 去重；"你吃饭了吗" vs "你吃饭了没" $\rightarrow$ 75%相似 $\rightarrow$ 去重。

Q16: 自定义 Capacitor 插件怎么实现?

1. **Java侧**: `@CapacitorPlugin(name="ZuiTiNotification")` + `@PluginMethod` 注解暴露方法
 2. **事件推送**: `notifyListeners("wechatMessage", data)` 推送WebView
 3. **JS侧**: `Plugin.addListener('wechatMessage', callback)` 监听
 4. **注册**: `MainActivity.onCreate` → `registerPlugin()`
 5. **配置注入**: 每次 `cap sync` 后运行 `patch-plugins.js` 注入自定义插件到 `capacitor.plugins.json`
-

Q17：6种反驳风格 + Mock模板量？

风格	模板数	特点
SHARP 犀利直击	8条	一针见血
HUMOROUS 幽默调侃	8条	笑里藏刀
PASSIVE 阴阳怪气	6条	表面客气
RATIONAL 理性分析	5条	逻辑拆解
GENTLE 温柔化解	5条	以柔克刚
DOMINEER 霸总护短	5条	强势维护

关键词检测正则（5类场景）：**催婚** /催|结婚|对象|找.*人|单身|恋爱/、**借钱** /钱|借|小气|穷|抠/、**工作** /工作|方案|项目|任务|加班/、**身材** /胖|身材|吃|外卖|健康/、**关系** /朋友|关系|感情|真心/

三、通用 / 跨项目问题

Q18: 两个项目最大挑战分别是什么?

- **宅家激励**: AI推荐响应速度 (API 2-5秒延迟) 。解决: 预加载天气+POI + 本地规则引擎兜底 + Loading动画
 - **嘴替**: 国产ROM后台杀进程。解决: 双通道 + 前台服务保活 (在小米/华为上复现并验证)
-

Q19: 为什么纯前端 + 无后端?

1. **隐私优先**: 用户数据不经第三方服务器
 2. **零成本**: 无服务器/域名/数据库费用
 3. **一人开发效率**: 纯前端减少30-50%工作量
 4. **局限性诚实承认**: 无法多设备同步、无法云端备份。用户量增长后考虑加Firebase
-

Q20：工程造价背景对做AI开发有什么帮助？

1. **系统思维**：建筑项目全局统筹（成本/工期/资源/风险）和软件架构方法论相通
 2. **成本敏感**：对API token消耗、降级策略、零服务器成本有天然意识
 3. **风险预案**：工程造价里的风险预备金 → AI应用的降级Fallback
 4. **从解决自身问题出发**：两个项目都来自真实需求，不是凭空造demo
-

Q21：产品化还需要做什么？

维度	宅家激励	嘴替
安全	API Key加密存储	API Key加密 + 隐私政策
体验	宠物AI生图替换emoji	多轮上下文优化
后端	Firebase云同步	云端对话历史
变现	会员订阅更多宠物/主题	按API调用量阶梯收费
合规	-	隐私政策+用户协议（涉消息读取）
测试	真机多Android版本适配	真机+国产ROM兼容性

Q22：你觉得这两个项目里最体现你能力的是哪个部分？

嘴替的 `sideEffects` 状态机。原因：

- 不是"用了什么框架"，而是自己设计了一套解耦方案
 - 体现了对复杂状态的建模能力
 - 面试时可以当场画状态图 + 讲设计思路
 - 和AI应用结合紧密（状态机管理LLM调用全生命周期）
-

面试话术速记

场景	要点
自我介绍	"工程造价应届生，独立开发了两款AI应用，从需求到APK全流程。一个做LLM对话，一个做AI Agent推荐"
为什么跨专业	"造价培养的是系统性思维+成本控制意识，这和设计AI应用的工程能力互补"
最有挑战的事	"国产ROM杀进程→双通道方案→小米华为真机验证，这是一个完整的debug→调研→方案→验证循环"
你能带来什么	"动手能力（两个完整产品）+ AI工程化思维（降级/容错/成本控制）+ 快速学习（从零到APK）"